The Last 5% Problem of Rewriting Software with AI

Recently, people are reverse-engineering and rewriting popular applications like Photoshop (PhotoCraft[1]) and Microsoft Word (WordCraft[2]) as open-source applications with the help of AI. There are a lot of discussions around this, and I wish to focus on the last 5% problem: even if an open-source rewrite reaches very high fidelity, there are still likely to be differences that cannot be easily eliminated. Those differences might become a problem when high fidelity is a requirement for the AI rewrite.

I happened to experiment with reverse-engineering and rewriting old code with AI earlier. The first case I want to share is my rewrite of the Excel 2000 **Dev Hunter** Easter egg game.[3] I previously wrote about it in *Paged Out!* issue 6. It is a hidden car-shooter game in Excel 2000. I wrote a small `loader.exe` in assembly language that loads `MSOWC.DLL` and runs the game. While it works, it is imperfect because you still have to carry the Excel 2000 DLL, and I cannot distribute that DLL due to copyright restrictions. So I wanted to reverse-engineer and rewrite the game as a standalone program.

I thought this would probably be an easy task, but it did not turn out to be that easy. If you look at Fig. 1, the image on the left is what I got on the first try. It is somewhat recognizable, but the image on the right, which is the final result, is very different. I prompted the AI to use the debugger as a reference to observe values and calculations, and then fix the rewrite. Eventually it reached a state I was very happy with. If you play it, it feels similar to the original.

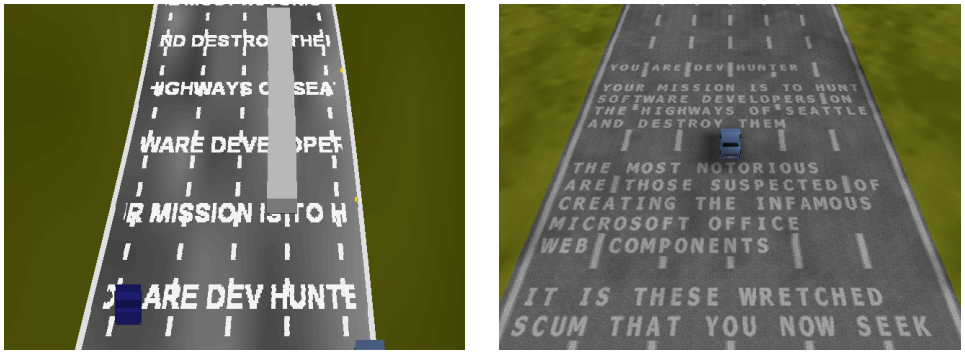


Figure 1: The first attempt (left) and the final result (right).

But when I looked at it more closely, I realized it was still missing two things. There is a rainy mode that appears with a certain probability, and the evening scene does not match exactly. None of these differences prevent this from being a very good remake of a childhood game I used to play. But it is still technically different from the original game.

Someone might argue that this does not really matter because you would not expect a game remake to be 100% accurate. However, another project I tried to reproduce is a little different. It is an old Chinese chess (Xiangqi) engine.[4] In that case, I did much the same thing: I asked the AI to analyze the program through static reverse engineering and with the help of a debugger, then rewrite it. It quickly got pretty close, but the remaining differences took a long time and a lot of tokens to figure out.

I set up a loop to generate random positions, run them through the original and rewritten engines, and compare their analyses. Whenever there was a difference, I asked the AI to find where it came from and fix it.[5]

Eventually, I was satisfied that the rewritten engine matched the original on 408,708 test cases. For my purposes, that is good enough. But consider a financial system or another critical system, where you want the rewrite to be exactly the same as the original. How do you approach that? Do you come up with a way to prove it? Do you use formal verification? Even if you test a million or a billion cases and they match, you cannot be sure that the two programs are necessarily the same. In general, there is no algorithm that can decide whether any two arbitrary programs are equivalent.[6]

It seems that AI can get a rewrite of an existing program started and bring it to a point that looks right or works well in practice. Getting it to 100% accuracy is a very different challenge.

---

[1] `https://github.com/storytold/photocraft`
[2] `https://github.com/storytold/wordcraft`
[3] `https://github.com/xusheng6/excel2000-devhunter-standalone`
[4] `https://github.com/xusheng6/cch-cms1`
[5] My longer account of this project: `https://xusheng.dev/posts/rebuilding-a-dos-xiangqi-engine/`.
[6] Program-equivalence discussion: `https://facultyweb.cs.wwu.edu/~wehrwes/courses/csci301_25s/lectures/L32/N32.html`